

CSE108 – Computer Programming Laboratory

Lab #8

Date: 17 April 2026

Part 0. Structural Requirements

- Distinct Functions: The recursive cost function must be a separate function. `main()` prompts for the object name and calls it.
- Array Size Limit: You cannot create any array larger than 101 elements.
- Input Limits: Max user input length is 100 characters. Max part name length is 20 characters. Max number of sub-parts per input line is 10.

Part 1 [100 pts] – Compound Object Cost Calculator

Background

A compound object is composed of smaller parts, some of which may also be compound objects. This hierarchy forms a tree: each node is a part and its children are its sub-parts. Leaf nodes are simple objects with a known unit price. The total cost is computed recursively by summing the costs of all parts. Figure 1 shows the structure of a bicycle as an example.

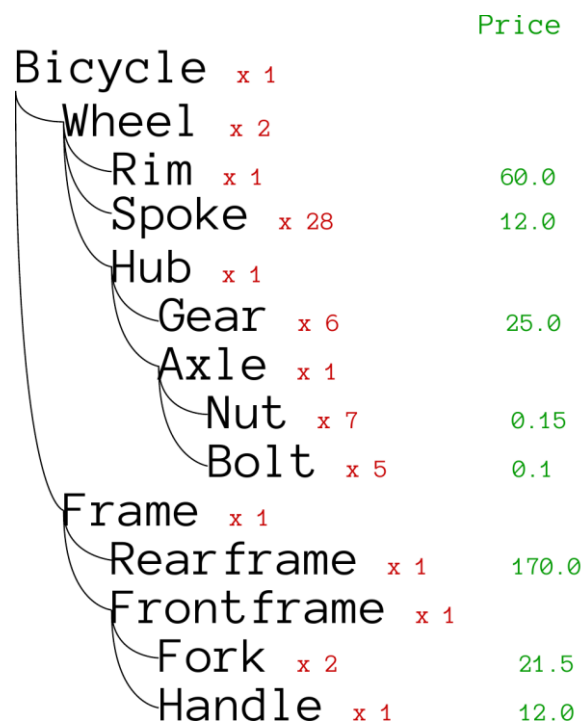


Figure 1: Composition of a bicycle

Scenario

Write an interactive C program that discovers the structure of a compound object at runtime by asking the user questions, then computes and prints its total cost. The program has no prior knowledge of the object.

When prompted to define a part, the user either enters a single float (unit price) or a list of (count, name) pairs for its sub-parts, e.g. 2 Wheel 1 Frame.

Suggested Function Prototype

The following prototype is a suggestion, not a requirement. You may design your own function signature as long as the program behaves correctly.

```
/* Recursively prompts the user to define 'name' and returns its total cost.
   'count' is how many of this part exist at the current level.          */
double compute_cost(char name[], int count, char parent[]);
```

Provided Helper: parse_line

You are given the following helper function to use as-is or adapt. It parses one line of user input and returns 1 if the line is a single number (simple part), or 2 if it contains (count, name) pairs (compound part).

```
#include <stdio.h>
#include <string.h>

#define MAX_NAME 21
#define MAX_PAIRS 10

/* Returns:
   1 -> single double (simple part, price stored in *number)
   2 -> count/name pairs (compound part, stored in values[] and labels[]) */
int parse_line(char *line,
               int values[MAX_PAIRS],
               char labels[MAX_PAIRS][MAX_NAME],
               int *count,
               double *number)
{
    *count = 0;
    char buffer[100];
    strcpy(buffer, line); /* strtok modifies the string */
    char *token = strtok(buffer, " \t");
    if (token == NULL) return 0;
    char *second = strtok(NULL, " \t");

    /* Case 1: single token -> double */
    if (second == NULL) {
        sscanf(token, "%lf", number);
        return 1;
    }

    /* Case 2: (count, name) pairs */
    while (token != NULL && second != NULL) {
        sscanf(token, "%d", &values[*count]);
        strncpy(labels[*count], second, MAX_NAME - 1);
        labels[*count][MAX_NAME - 1] = '\0';
        (*count)++;
        token = strtok(NULL, " \t");
        second = strtok(NULL, " \t");
    }
    return 2;
}
```

Logic

- Print a prompt such as `Define Wheel in Bicycle?:`
- Read a full input line with `fgets()` (max 100 chars).
- Call `parse_line()`. If it returns 1, this is a simple part – return `count * price`.
- If it returns 2, recursively call `compute_cost()` for each (count, name) pair and accumulate the results.
- Return the accumulated total multiplied by the count of the current part.
- Do not store the full tree in memory – compute costs on-the-fly as the user provides input.

Example Interaction

```
> Name the object:
> Bicycle
> Define Bicycle?:
> 2 Wheel 1 Frame
> Define Wheel in Bicycle?:
> 1 Rim 28 Spoke 1 Hub
> Define Rim in Wheel?:
> 60
> Define Spoke in Wheel?:
> 12.0
> Define Hub in Wheel?:
> 6 Gear 1 Axle
> Define Gear in Hub?:
> 25.0
> Define Axle in Hub?:
> 7 Nut 5 Bolt
> Define Nut in Axle?:
> 0.15
> Define Bolt in Axle?:
> 0.1
> Define Frame in Bicycle?:
> 1 Rearframe 1 Frontframe
> Define Rearframe in Frame?:
> 170.0
> Define Frontframe in Frame?:
> 2 Fork 1 Handle
> Define Fork in Frontframe?:
> 21.5
> Define Handle in Frontframe?:
> 12
> Total cost of Bicycle is 1320.1
```

Hints

- The provided `parse_line()` handles tokenisation for you – you do not need to implement your own string parser from scratch.
- Input may contain multiple spaces between tokens; the helper already handles this.
- Test with objects other than the bicycle (e.g. a car engine, a meal).